

Hoshi Frontend Jenkins Pipeline

1. Scenario

The Hoshi frontend is a Vite application deployed locally through Apache. The source is maintained in:

```
/home/neural/workarea/Aaditya/software/hoshi/hoshi-3.0-b3bb8369
```

Jenkins copies that source into its isolated workspace, installs exact dependencies, runs all tests, builds the application, and publishes the generated `dist` directory to:

```
/var/www/hoshi-frontend
```

Apache serves the deployed application at:

```
http://localhost/
```

2. Initial Problems and Root Causes

2.1 Node.js Incompatibility

The application had previously worked with backed-up dependencies, but a fresh `npm install` and build failed because the active Node.js release was not compatible with the current Vite/Rolldown toolchain.

The project and Jenkins environment were standardized to:

```
Node.js 22.18.0  
npm 10.9.3
```

The required version is recorded in `.nvmrc`, while `package.json` defines supported Node versions:

```
"engines": {  
  "node": "^20.19.0 || ^22.13.0 || >=24.0.0"  
}
```

This ensures local development and CI use compatible Node versions and unsupported environments fail early.

2.2 Jenkins Access

Jenkins runs as the **jenkins** Linux user rather than the developer account.

Therefore it required:

- Read and directory traversal access to the project source
- Write access to `/var/www/hoshi-frontend`
- A configured Jenkins NodeJS installation named **Node 22**

Permissions were verified using:

```
sudo -u jenkins test -r  
/home/neural/workarea/Aaditya/software/hoshi/hoshi-3.0-b3bb8369/package.json \  
&& echo "Jenkins can read the source"
```

```
sudo -u jenkins test -w /var/www/hoshi-frontend \  
&& echo "Jenkins can deploy"
```

Jenkins runtime verification:

```
whoami  
command -v node  
command -v npm  
node --version  
npm --version
```

Output confirmed:

```
Node v22.18.0  
npm 10.9.3
```

2.3 Test Failures

The first complete execution reported **38 failed tests**.

Most failures were outdated UI expectations, while others revealed genuine application issues.

Fixes included:

- Providing tooltip context in notification tests
- Updating delivery-channel tests for the latest UI
- Correcting notification channel mapping
- Fixing log row interactions
- Improving accessibility for shared table search and sorting
- Updating roster, organogram, template, filter, readiness, and event assertions

Final results:

Test files/suites: 357 passed, 0 failed

Tests: 1,253 passed, 0 failed

Production build: Successful

3. Jenkins Prerequisites

Install and configure:

1. Jenkins
2. Jenkins NodeJS Plugin
3. NodeJS tool named **Node 22** configured as **Node 22.18.0**
4. `rsync`
5. `curl`
6. Apache with `mod_alias` (or standard VirtualHost support)
7. Source read and deployment write permissions for the Jenkins user

Example deployment directory setup:

```
sudo mkdir -p /var/www/hoshi-frontend
sudo chown -R jenkins:www-data /var/www/hoshi-frontend
sudo chmod -R 775 /var/www/hoshi-frontend
```

4. Apache Configuration

Configuration file:

```
/etc/apache2/sites-available/hoshi-frontend.conf
<VirtualHost *:80>
    ServerName localhost
    DocumentRoot /var/www/hoshi-frontend
```

```
<Directory /var/www/hoshi-frontend>
  Options -Indexes +FollowSymLinks
  AllowOverride None
  Require all granted
  FallbackResource /index.html
</Directory>

ErrorLog ${APACHE_LOG_DIR}/hoshi-frontend-error.log
CustomLog ${APACHE_LOG_DIR}/hoshi-frontend-access.log combined
</VirtualHost>
```

`FallbackResource /index.html` ensures SPA routes such as:

`/employees`

are handled by the frontend instead of returning an Apache 404.

Enable the site:

```
sudo a2dissite 000-default.conf
sudo a2ensite hoshi-frontend.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
sudo systemctl is-active apache2
```

Warnings about PHPMyAdmin aliases or global `ServerName` can be ignored as long as:

Syntax OK

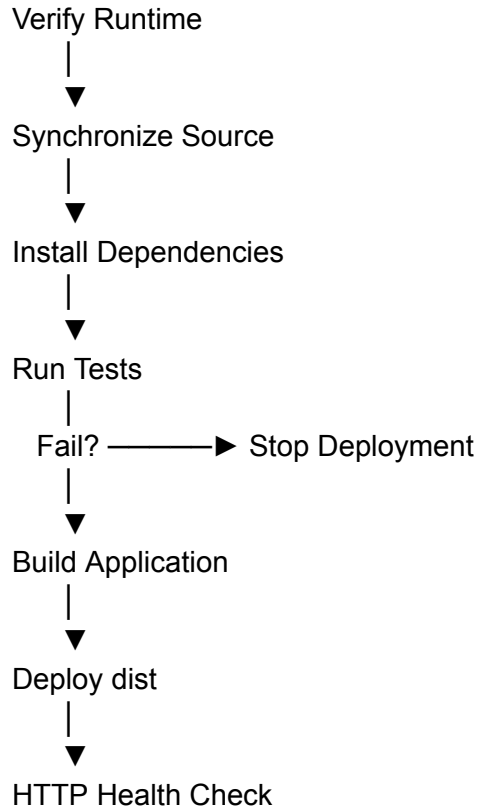
is reported.

5. Jenkins Job Setup

1. Create a **Pipeline** job named **hoshi-frontend**.
2. Under **Pipeline**, either:
 - Choose **Pipeline script from SCM**, or
 - Paste the repository's **Jenkinsfile**.
3. Save the job.
4. Click **Build Now**.

Using **Pipeline Script from SCM** is recommended because Jenkins automatically tracks changes to the repository's **Jenkinsfile**.

6. Pipeline Flow



Stage 1 – Verify Runtime

Jenkins displays:

- Current user
- Node executable
- npm executable
- Node version
- npm version

A Node script validates the runtime before installation begins.

Stage 2 – Synchronize Source

The project is copied into the Jenkins workspace:

```
rsync -a --delete --delete-excluded [exclusions] "$SOURCE_DIR/" "$WORKSPACE/"
```

Excluded items include:

- `.git`
- `node_modules`
- Previous builds
- Reports
- Backup files
- Logs
- Scratch files

`--delete` removes workspace files that no longer exist in the source.

Stage 3 – Install Dependencies

```
npm ci --no-audit --no-fund
```

`npm ci`:

- Uses `package-lock.json`
 - Deletes previous dependencies
 - Produces deterministic installations
 - Does not modify the lock file
-

Stage 4 – Test Gate

```
npm test
```

Vitest executes once using:

```
vitest run
```

Any failed test immediately stops the pipeline.

No build or deployment occurs unless every test passes.

Stage 5 – Build

`npm run build`

Vite generates the optimized production application inside:

`$WORKSPACE/dist`

Chunk-size warnings indicate optimization opportunities but are **not** build failures.

Stage 6 – Local Deployment

The pipeline first verifies:

- `dist/index.html` exists
- Jenkins has deployment permissions

Deployment is performed using:

```
rsync -a --delete "$WORKSPACE/dist/" /var/www/hoshi-frontend/
```

Only generated production files are deployed.

Stage 7 – Deployment Verification

```
curl --fail --silent --show-error http://127.0.0.1/ >/dev/null
```

The pipeline completes successfully only if Apache returns a successful HTTP response.

7. Safety and Reliability Controls

The pipeline includes several safeguards:

- Prevents concurrent builds
- Retains only the latest 10 builds
- Adds timestamps to logs
- Applies a 20-minute timeout
- Uses `set -eu` to stop on errors or unset variables

- Validates required files before copying
-

8. Normal Operating Procedure

After modifying frontend code:

1. Optionally run:
 - `nvm use`
 - `npm test`
 - `npm run build`
2. Trigger the **hoshi-frontend** Jenkins job.
3. Verify:
 - Runtime validation
 - Test execution
 - Build
 - Deployment
 - Health check
4. Open:

`http://localhost/`

and perform a quick UI smoke test.

Useful diagnostics:

```
sudo systemctl status jenkins
sudo systemctl status apache2
sudo tail -f /var/log/apache2/hoshi-frontend-error.log
curl -I http://127.0.0.1/
```

9. Concise Interview Explanation

I built a Jenkins CI/CD pipeline for a Vite frontend deployed through Apache. The initial fresh build failed because the local and Jenkins Node versions were incompatible with the Vite/Rolldown toolchain. I standardized both environments on Node.js 22.18.0 and documented the requirement in `.nvmrc` and `package.json`. Since Jenkins runs as its own Linux user, I verified source-read and deployment-write permissions. The pipeline synchronizes a clean source copy

using `rsync`, performs a deterministic `npm ci`, and runs the complete Vitest suite as a deployment gate. Only after all 1,253 tests pass does it build the application and synchronize the generated `dist` directory to Apache's document root. Finally, the pipeline performs a `curl` health check against the deployed application, ensuring the deployment is successful only when the application is actually reachable. Additional safeguards include disabling concurrent builds, retaining build history, and applying execution timeouts.